

Clase 2. REST avanzado: diseño de recursos y versionamiento

Objetivo: Diseñar y documentar APIs RESTful robustas y escalables dentro del marco de una arquitectura orientada a servicios.

2.1. Relación entre SOA y REST: interoperabilidad y desacoplamiento

La transición de SOA (Service-Oriented Architecture) hacia REST (Representational State Transfer) no representó una ruptura total, sino una evolución natural de los principios de integración orientada a servicios. Ambas arquitecturas buscan interoperabilidad, modularidad y reutilización, pero lo hacen con diferentes enfoques tecnológicos y grados de acoplamiento.

1) De SOA a REST: una evolución conceptual

SOA definió la base de los sistemas distribuidos modernos al promover la idea de que las funcionalidades empresariales debían encapsularse como servicios independientes, comunicados mediante contratos formales (WSDL) y orquestados por un ESB (Enterprise Service Bus). REST, en cambio, simplificó este paradigma al utilizar directamente los principios de la web, donde cada recurso se identifica por una URI y se manipula mediante operaciones estándar HTTP.

Aspecto	SOA	REST
Contrato	WSDL/XML	OpenAPI/JSON
Transporte	SOAP/HTTP	HTTP puro
Arquitectura	Centralizada (ESB)	Descentralizada (API Gateway)
Estilo	RPC o Documento	Recurso (Resource-Oriented)
Escalabilidad	Vertical	Horizontal
Complejidad	Alta	Baja

REST no reemplaza a SOA, sino que hereda sus principios fundamentales (contratos, modularidad y reutilización), reduciendo la complejidad técnica y aumentando la escalabilidad.

2) Interoperabilidad: el lenguaje común entre servicios

La interoperabilidad es la capacidad de distintos sistemas —independientemente de sus plataformas, lenguajes o versiones— para intercambiar información de forma comprensible y segura.

Factores que promueven interoperabilidad en REST

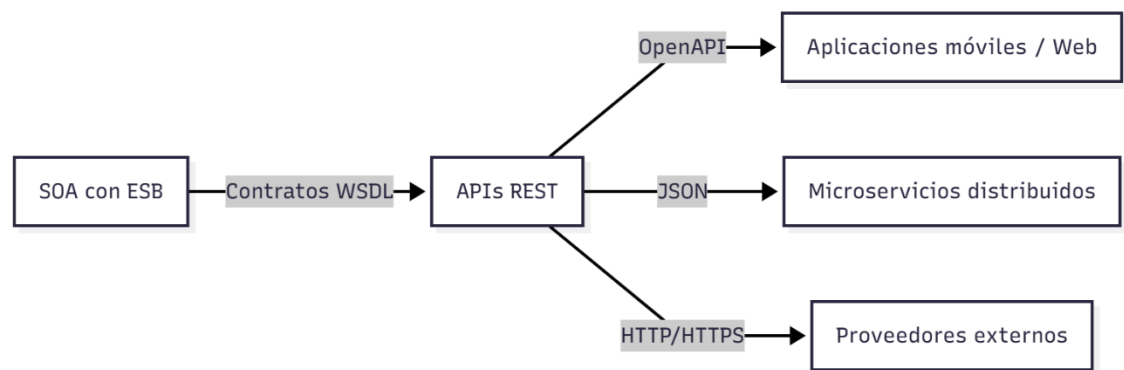
- Uso de protocolos estándar: HTTP/HTTPS, accesible desde cualquier lenguaje.
- Formatos de datos universales: JSON, XML, YAML.
- Contratos abiertos: OpenAPI/Swagger, legibles por humanos y máquinas.
- Uso de códigos de estado HTTP estandarizados para comunicación semántica.

Ejemplo de interoperabilidad REST

Un sistema de inventario en Java puede comunicarse con una aplicación móvil en Flutter o un microservicio en Node.js sin requerir adaptadores específicos, siempre que ambos respeten el contrato definido.

```
GET /productos/15
{
  "id": 15,
  "nombre": "Laptop Lenovo",
  "precio": 950.00,
  "moneda": "USD"
}
```

Imagen 1. Relación entre interoperabilidad y desacoplamiento



3) Desacoplamiento: la clave de la escalabilidad

El desacoplamiento se refiere a la capacidad de un sistema para evolucionar sin afectar a otros. REST logra esto gracias a su enfoque stateless y al uso de recursos independientes, lo que permite que cada API funcione de manera autónoma.

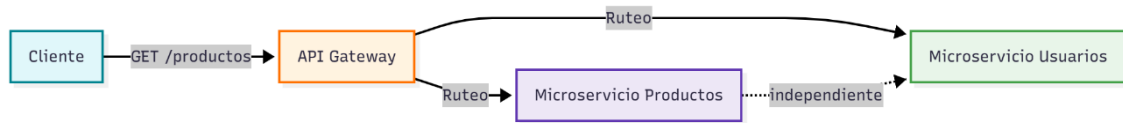
Niveles de desacoplamiento

1. **Tecnológico:** clientes y servidores pueden estar escritos en diferentes lenguajes.
2. **Temporal:** un servicio puede ser actualizado sin interrumpir a los consumidores.
3. **Funcional:** cada recurso tiene una única responsabilidad.

Ejemplo comparativo

Escenario	SOA	REST
Comunicación	XML/SOAP	JSON/HTTP
Dependencia temporal	Alta (ESB central)	Baja (endpoints directos)
Cambios en servicios	Requieren redistribución de contratos	Versionamiento simple (v1, v2)
Escalado	Limitado por el ESB	Escalado independiente de cada servicio

Imagen 2. Diagrama de desacoplamiento REST



La evolución de un microservicio (por ejemplo, añadir un nuevo campo o endpoint) no afecta al resto del sistema si el contrato mantiene compatibilidad.

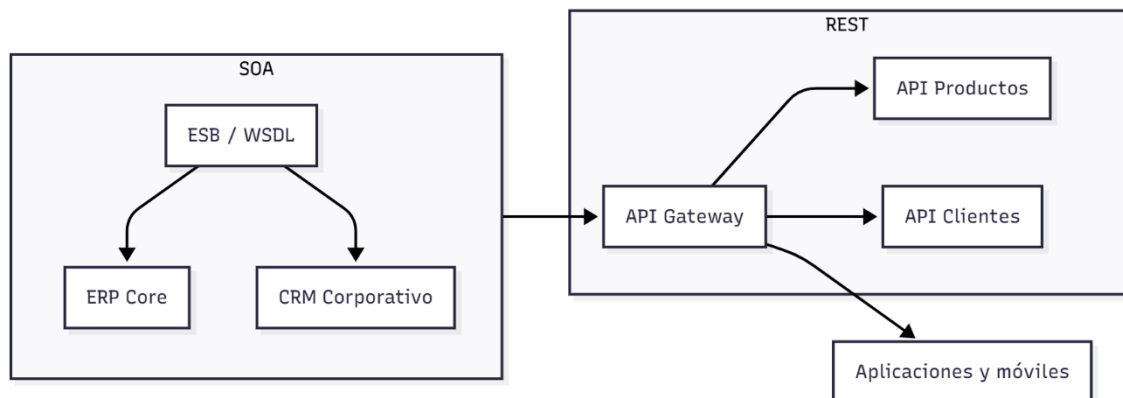
4) Complementariedad entre SOA y REST en arquitecturas híbridas

Muchas empresas modernas no reemplazan totalmente su infraestructura SOA, sino que la extienden con REST para habilitar nuevas funcionalidades.

Ejemplo práctico

1. Los sistemas core (ERP, CRM) permanecen en SOA con servicios SOAP.
2. Se implementa una capa de APIs REST que expone funcionalidades simplificadas.
3. El API Gateway actúa como traductor entre el mundo SOAP (WSDL) y REST (OpenAPI).

Imagen 3. Arquitectura híbrida SOA + REST



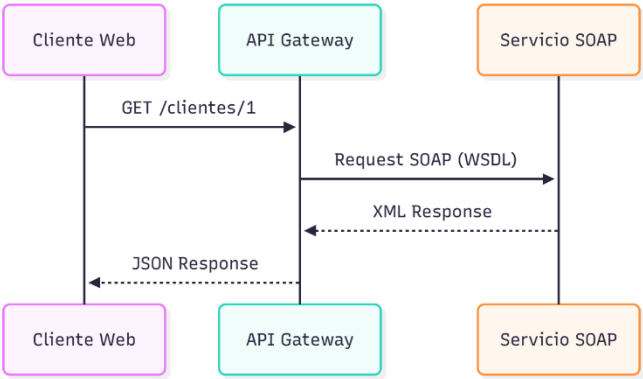
La organización mantiene la estabilidad del núcleo SOA mientras aprovecha la flexibilidad y rapidez del modelo REST.

5) Patrones comunes de integración entre SOA y REST

Patrón	Descripción	Ejemplo
Wrapper Pattern	Se crea una capa REST encima de servicios SOAP antiguos.	API REST → traduce → SOAP WSDL
Anti-Corruption Layer (ACL)	Separa las lógicas de negocio de sistemas legados.	Microservicio REST intermedio entre ESB y frontend
Gateway Routing	El API Gateway enruta peticiones REST a servicios SOAP internos.	Kong / WSO2 Gateway ↔ Mule ESB

Adapter Pattern	Adapta mensajes entre formatos XML y JSON.	Adapter transforma XML → JSON antes de entregar la respuesta
------------------------	--	--

Imagen 4. Flujo Wrapper REST sobre SOAP



6) Beneficios y desafíos de la integración SOA–REST

Categoría	Beneficio	Desafío
Desacoplamiento	Actualizaciones independientes	Coordinación entre contratos
Interoperabilidad	Comunicación multiplataforma	Transformaciones entre formatos
Seguridad	OAuth2 y WS-Security coexistentes	Gestión de tokens y certificados
Gobernanza	Políticas unificadas en Gateway	Complejidad en control de versiones
Desempeño	Reducción de latencia	Balanceo y caching adecuados

Claves del éxito:

- Usar API Gateways con conectores SOAP ↔ REST.
- Mantener un catálogo de contratos (WSDL + OpenAPI).
- Implementar observabilidad unificada (logs, métricas, trazas).
- Automatizar pruebas de interoperabilidad y conversión.

2.2. Principios REST y HATEOAS

El diseño de una API RESTful no se limita a exponer endpoints HTTP. Su verdadero valor radica en seguir los principios arquitectónicos definidos por Roy Fielding (2000), que garantizan escalabilidad, simplicidad, interoperabilidad y evolución sin ruptura. Además, el principio de HATEOAS (Hypermedia As The Engine Of Application State) añade una capa de autodocumentación que hace que las APIs sean descubribles y autodescriptivas.

1) Fundamentos de REST

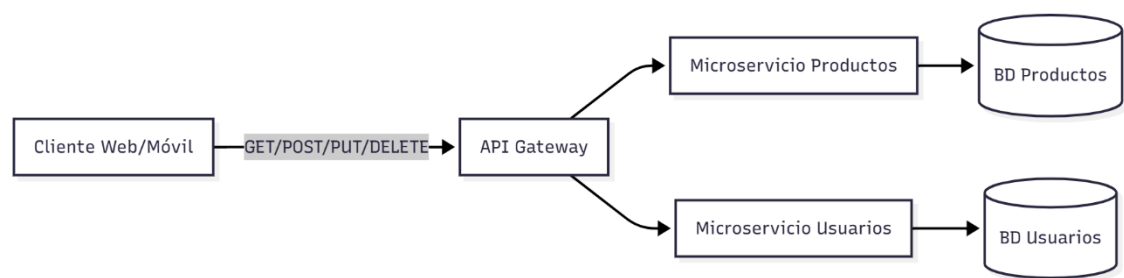
REST (Representational State Transfer) es un estilo arquitectónico para sistemas distribuidos basado en la web. Propone un conjunto de restricciones que, al cumplirse, otorgan las propiedades de desempeño, confiabilidad y escalabilidad que caracterizan a Internet.

Restricciones principales

Restricción	Descripción	Beneficio
Cliente-Servidor	Separación entre interfaz (cliente) y almacenamiento/procesamiento (servidor).	Independencia, escalabilidad y reusabilidad.
Sin estado (Stateless)	Cada solicitud contiene toda la información necesaria; el servidor no mantiene sesiones.	Escalabilidad y simplicidad.
Cacheable	Las respuestas pueden ser almacenadas para optimizar rendimiento.	Reducción de latencia y carga de red.
Interfaz uniforme	Uso coherente de URIs, verbos y formatos de mensaje.	Interoperabilidad y claridad.
Sistema en capas	Los componentes intermedios (gateways, proxies) pueden intervenir sin afectar al cliente.	Seguridad, caching y balanceo de carga.
Código bajo demanda (opcional)	El servidor puede enviar código ejecutable (JavaScript) al cliente.	Flexibilidad en clientes ligeros.

Estas restricciones conforman la base de las buenas prácticas RESTful. Cuantos más principios adopta una API, más cercana estará al ideal REST.

Imagen 5. Arquitectura RESTful básica



2) El modelo de madurez de Richardson

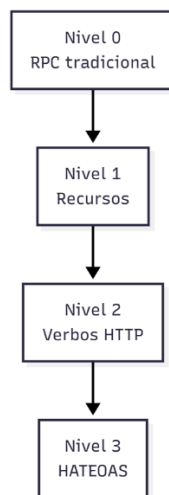
El modelo de Leonard Richardson clasifica las APIs REST según su nivel de cumplimiento de los principios REST. Es útil para evaluar el grado de madurez de una API.

Nivel	Descripción	Ejemplo
Nivel 0: RPC	Llamadas sin estructura uniforme (SOAP, XML-RPC).	/getClient?id=1

Nivel 1: Recursos	Se introducen URIs para representar entidades.	/clientes/1
Nivel 2: Verbos HTTP	Uso correcto de métodos (GET, POST, PUT, DELETE).	GET /clientes/1
Nivel 3: HATEOAS	Respuestas con hipervínculos que guían al cliente.	{ "_links": { "self": "/clientes/1" } }

Una API verdaderamente RESTful alcanza el nivel 3, donde las operaciones son autodescriptivas y el cliente puede navegar los recursos dinámicamente.

Imagen 6. Escalera de madurez de Richardson



3) HATEOAS: navegabilidad y autodescripción

HATEOAS (Hypermedia As The Engine Of Application State) es la pieza final del rompecabezas REST. Permite que una API se autodescriba: el cliente no necesita conocer de antemano todos los endpoints, porque el servidor le indica las posibles acciones disponibles a través de enlaces hipertexto (*links*).

Ejemplo de respuesta con HATEOAS

```

{
  "id": 101,
  "nombre": "Laptop Dell",
  "precio": 1200,
  "stock": 20,
  "_links": {
    "self": { "href": "/productos/101" },
    "actualizar": { "href": "/productos/101", "method": "PUT" },
    "eliminar": { "href": "/productos/101", "method": "DELETE" },
    "relacionados": { "href": "/productos?categoria=laptops" }
  }
}

```

Con HATEOAS, las APIs se comportan como sitios web navegables, donde cada respuesta indica las acciones posibles. Esto reduce el acoplamiento y facilita la evolución sin romper clientes.

Beneficios principales

- **Descubrimiento dinámico:** los clientes descubren nuevos recursos sin documentación adicional.
- **Desacoplamiento semántico:** los cambios en endpoints no rompen integraciones.
- **Autodescripción:** cada recurso incluye su contexto y acciones.
- **Evolución controlada:** se pueden añadir enlaces sin romper versiones previas.

Imagen 7. Navegación RESTful con HATEOAS



4) Diseño semántico y recursos en REST

El poder de REST está en el modelado correcto de los recursos. Cada recurso debe representar una entidad de negocio o proceso identificable, accesible mediante una URI única.

Buenas prácticas de diseño

Recomendación	Ejemplo correcto	Ejemplo incorrecto
Usar sustantivos en plural	/clientes, /pedidos	/getCliente, /crearPedido
Representar jerarquías	/clientes/5/pedidos	/pedidoPorCliente?id=5
Usar filtros como parámetros	/productos?categoria=laptops&page=2	/productos/laptops/pagina2
Evitar acciones en el URI	POST /pedidos	/procesarPedido
Versionar explícitamente	/api/v1/clientes	/clientes sin versión

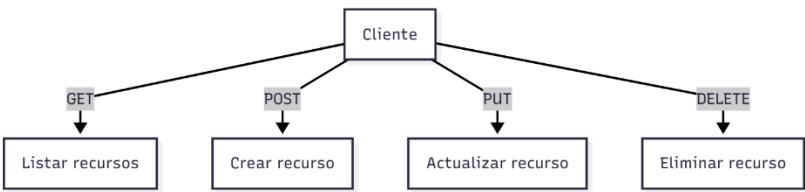
Tip: cada recurso debe ser autónomo, versionado y autocontenido, incluyendo enlaces y metadatos relevantes.

5) Códigos de estado HTTP y semántica uniforme

Una API RESTful usa los **códigos de estado HTTP** como lenguaje de comunicación universal.

Código	Significado	Uso típico
200 OK	Operación exitosa	Respuesta a GET o PUT
201 Created	Recurso creado	Respuesta a POST
204 No Content	Sin cuerpo de respuesta	DELETE exitoso
400 Bad Request	Petición inválida	Error de formato o validación
401 Unauthorized	Requiere autenticación	Token ausente o inválido
403 Forbidden	Acceso denegado	Rol sin permisos
404 Not Found	Recurso no existe	URI incorrecto
409 Conflict	Estado inconsistente	Duplicado o error de concurrencia
500 Internal Server Error	Falla del servidor	Excepción no controlada

Imagen 8. Flujo CRUD y códigos HTTP



6) Ejemplo práctico: API de pedidos

Un ejemplo integrador de los principios REST y HATEOAS podría lucir así:

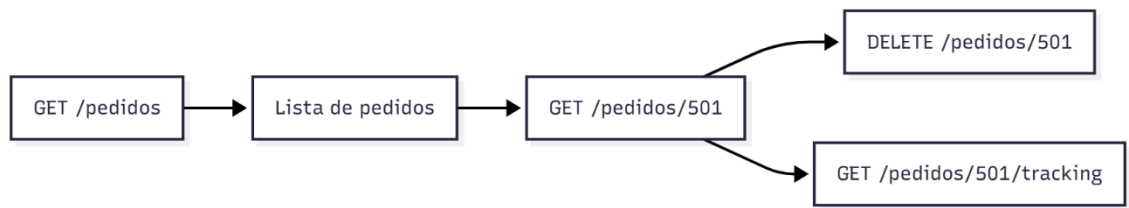
Solicitud de recurso

GET /pedidos/501
Accept: application/json

Respuesta

```
{
  "id": 501,
  "cliente": "/clientes/33",
  "monto": 120.50,
  "estado": "Enviado",
  "_links": {
    "self": { "href": "/pedidos/501" },
    "cancelar": { "href": "/pedidos/501", "method": "DELETE" },
    "seguimiento": { "href": "/pedidos/501/tracking" }
  }
}
```

Imagen 9. Diagrama de navegación de pedidos REST



7) Beneficios y desafíos de aplicar REST + HATEOAS

Categoría	Beneficio	Desafío
Interoperabilidad	Compatible con cualquier tecnología HTTP	Definir contratos consistentes
Escalabilidad	Sin estado, permite balanceo	Gestión de sesiones distribuida
Evolución	Navegación dinámica mediante enlaces	Implementación compleja en clientes
Mantenibilidad	Contratos versionados y legibles	Requiere disciplina documental

Aplicar REST junto con HATEOAS genera APIs autodescriptivas, estandarizadas y sostenibles, reduciendo la dependencia entre equipos y favoreciendo la interoperabilidad en entornos empresariales.

2.3. Diseño de URIs, verbos HTTP y códigos de estado

El diseño de URIs, el uso correcto de los verbos HTTP y la gestión de los códigos de estado son los pilares de una API RESTful robusta. Estos elementos conforman la interfaz uniforme que garantiza consistencia, interoperabilidad y facilidad de uso. En entornos empresariales complejos, seguir estándares REST reduce errores de integración, mejora la mantenibilidad y acelera la adopción por parte de terceros.

1) Principios para el diseño de URIs

Una URI (Uniform Resource Identifier) representa la dirección única de un recurso dentro del sistema. En REST, cada URI debe expresar una entidad de negocio (cliente, producto, pedido, usuario) y ser semántica, estable y predecible.

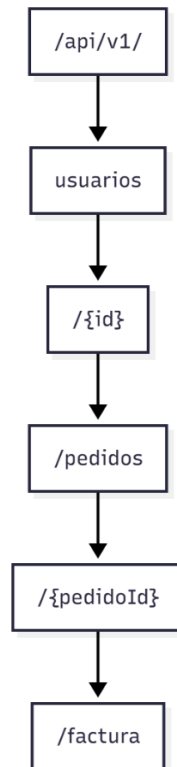
Buenas prácticas generales

Recomendación	Correcto	Incorrecto
Usar sustantivos en plural	/productos, /usuarios	/getProducto, /crearUsuario
Evitar verbos en la ruta	POST /pedidos	/procesarPedido
Mantener jerarquías lógicas	/usuarios/45/pedidos/3	/pedidosPorUsuario?id=45

No incluir extensiones de formato	/productos	/productos.json
Ser consistente en el estilo	/api/v1/productos	/Api/v1/Productos

Una buena URI debe poder leerse como una frase y entenderse sin necesidad de documentación adicional.

Imagen 10. Jerarquía de URIs RESTful



Ejemplo práctico:

GET /api/v1/usuarios/32/pedidos/155/factura

Traducción: obtener la factura del pedido 155 del usuario 32.

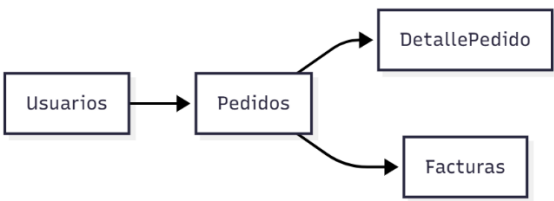
Paginación y filtros

Las APIs deben permitir consultas eficientes sin sobrecargar el servidor.

GET /api/v1/productos?categoria=laptops&page=2&limit=20&sort=precio_desc

Los parámetros de consulta (*query params*) permiten controlar el número de resultados, filtros y ordenamiento.

Imagen 11. Ejemplo de recursos relacionados



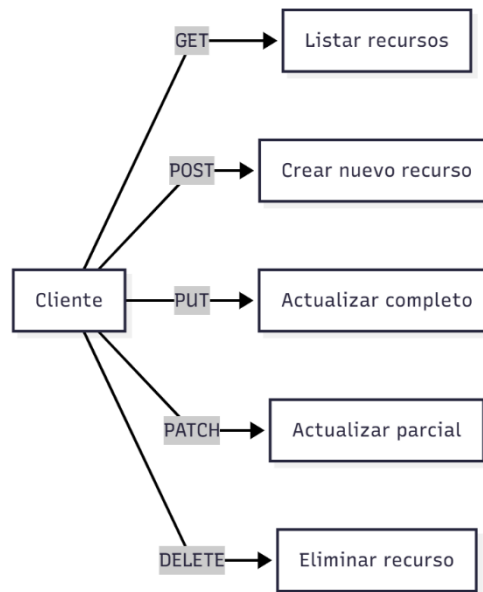
2) Verbos HTTP: operaciones universales

REST utiliza los verbos HTTP estándar como operaciones CRUD (Create, Read, Update, Delete) sobre los recursos. Comprender su semántica y aplicarlos correctamente evita ambigüedades.

Verbo	Operación	Idempotencia	Ejemplo	Descripción
GET	Leer	Sí	GET /productos	Recupera información. No modifica estado.
POST	Crear	No	POST /productos	Crea un nuevo recurso.
PUT	Actualizar (reemplazar)	Sí	PUT /productos/10	Reemplaza un recurso completo.
PATCH	Actualizar (parcial)	Sí	PATCH /productos/10	Modifica atributos específicos.
DELETE	Eliminar	Sí	DELETE /productos/10	Elimina un recurso existente.
HEAD	Cabecera	Sí	HEAD /productos	Devuelve metadatos sin cuerpo.
OPTIONS	Descubrimiento	Sí	OPTIONS /productos	Indica operaciones disponibles.

Idempotencia: ejecutar varias veces la misma operación no debe cambiar el resultado final (p. ej., borrar un registro ya borrado sigue devolviendo 204 No Content).

Imagen 12. Diagrama de flujo CRUD REST



Ejemplo práctico: API de pedidos

```
POST /api/v1/pedidos
{
  "clienteId": 20,
  "monto": 125.50
}
```

Respuesta:

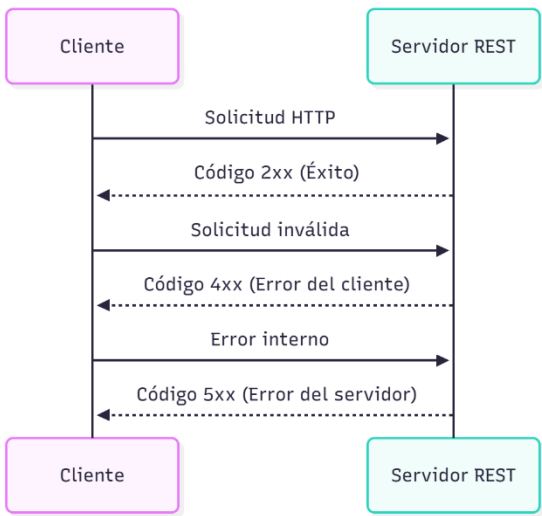
```
HTTP/1.1 201 Created
Location: /api/v1/pedidos/501
```

3) Códigos de estado HTTP

Los códigos de estado HTTP son el lenguaje común entre cliente y servidor. Transmiten el resultado de una operación sin necesidad de analizar el cuerpo de la respuesta.

Categoría	Código	Significado	Ejemplo de uso
2xx Éxito	200 OK	Solicitud procesada correctamente.	GET/PUT exitoso.
	201 Created	Recurso creado.	POST exitoso.
	204 No Content	Operación exitosa sin contenido.	DELETE correcto.
3xx Redirección	301 Moved Permanently	Recurso movido.	Cambio de ruta permanente.
	304 Not Modified	Recurso sin cambios.	Cache válido.
4xx Error del cliente	400 Bad Request	Datos inválidos.	Faltan campos o formato incorrecto.
	401 Unauthorized	Falta autenticación.	Token ausente.
	403 Forbidden	Sin permisos.	Usuario sin rol adecuado.
	404 Not Found	Recurso inexistente.	ID inválido.
	409 Conflict	Estado inconsistente.	Duplicado o error de concurrencia.
5xx Error del servidor	500 Internal Server Error	Falla no controlada.	Excepción en backend.
	503 Service Unavailable	Servidor sobrecargado o en mantenimiento.	Pausa temporal.

Imagen 13. Ciclo de respuesta HTTP



4) Estructura de respuestas y manejo de errores

Una API profesional debe proporcionar respuestas estructuradas y coherentes, tanto en casos de éxito como de error.

Respuesta exitosa (GET)

```
{
  "status": 200,
  "message": "Operación exitosa",
  "data": [{ "id": 1, "nombre": "Laptop Lenovo" }]
}
```

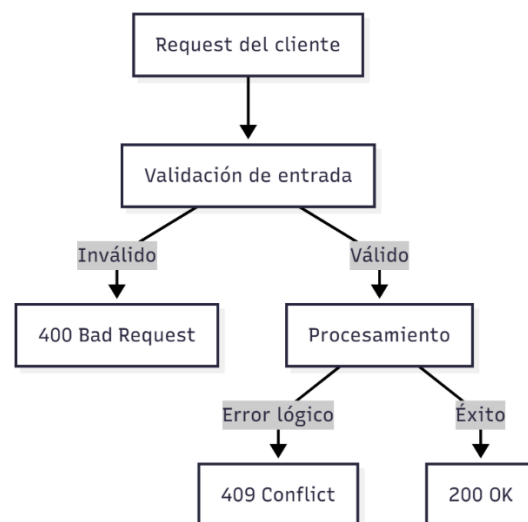
Respuesta de error (400 Bad Request)

```
{
  "status": 400,
  "error": "Datos inválidos",
  "details": ["El campo 'precio' es requerido"]
}
```

Recomendaciones:

- Mantener estructura uniforme (status, message, data o error).
- Usar nombres en inglés si la API es pública.
- Incluir códigos de error internos (error_code) para diagnósticos.

Imagen 14. Flujo de manejo de errores



5) Versionamiento de URIs

El versionamiento garantiza que una API pueda evolucionar sin romper clientes existentes. Se recomienda usar la convención `/api/v1/` o cabeceras personalizadas.

Ejemplo

GET /api/v1/productos
GET /api/v2/productos

Diferencias entre versiones pueden incluir nuevos campos o cambios en el formato de respuesta.

Buena práctica: mantener soporte para al menos una versión anterior mientras se comunica la deprecación mediante cabeceras:

Deprecation: true
Sunset: Sat, 31 Dec 2025 23:59:59 GMT

6) Tabla resumen de buenas prácticas RESTful

Categoría	Recomendación	Impacto
URIs	Semánticas, en plural, versionadas	Claridad y estabilidad
Verbos	Uso correcto de HTTP	Interoperabilidad
Códigos	Estandarizados (2xx, 4xx, 5xx)	Diagnóstico rápido
Errores	Mensajes consistentes y detallados	Facilidad de depuración
Versionamiento	Rutas o cabeceras claras	Evolución controlada

2.4. Versionamiento y compatibilidad hacia atrás

En una API REST, el versionamiento es una práctica esencial para garantizar su evolución controlada, permitiendo agregar nuevas funcionalidades o modificar comportamientos sin interrumpir el funcionamiento de los clientes existentes. La compatibilidad hacia atrás (backward compatibility) asegura que los consumidores que aún usan versiones anteriores sigan operando correctamente mientras adoptan gradualmente las nuevas.

1) Importancia del versionamiento

A medida que las APIs crecen, los requerimientos cambian: se agregan campos, se renombra una propiedad o se reemplaza una lógica. Si no existe una estrategia de versionamiento clara, cualquier cambio podría romper integraciones existentes, generando errores en producción y pérdida de confianza.

Objetivos del versionamiento:

- Mantener la estabilidad contractual entre proveedor y consumidor.
- Facilitar la evolución incremental de los servicios.
- Permitir una migración progresiva a nuevas versiones.
- Soportar paralelismo de versiones durante un tiempo definido.

Imagen 15. Evolución segura de una API versionada



2) Estrategias comunes de versionamiento

Existen diversas formas de identificar la versión de una API. La elección depende del contexto técnico, las políticas corporativas y las herramientas disponibles.

a) Versionamiento en la URL

Es la forma más simple y visible para los consumidores.

```
GET /api/v1/productos
GET /api/v2/productos
```

Ventajas:

- Claridad y simplicidad.
- Fácil de rastrear en logs y documentación

Desventajas:

- Puede generar duplicación de rutas.
- Dificulta la coexistencia entre microservicios interdependientes.

b) Versionamiento en cabeceras HTTP

La versión se comunica mediante el encabezado `Accept` o un encabezado personalizado.

```
GET /productos
Accept: application/vnd.api+json;version=1.0
```

Ventajas:

- Mantiene las URIs limpias.
- Permite múltiples versiones sobre la misma ruta.

Desventajas:

- Menor visibilidad para el usuario.
- Puede requerir configuración adicional en proxies o gateways.

c) Versionamiento por parámetro de query

Incluye la versión en la cadena de consulta.

GET /productos?version=2

Ventajas: fácil de implementar.

Desventajas: menos formal y menos adecuada para APIs públicas.

d) Versionamiento semántico (SemVer)

Basado en el formato MAJOR.MINOR.PATCH (por ejemplo, 2.1.4):

- **MAJOR:** cambios incompatibles.
- **MINOR:** nuevas funcionalidades compatibles.
- **PATCH:** correcciones de errores sin alterar contratos.

Imagen 16. Versionamiento semántico



3) Compatibilidad hacia atrás (Backward Compatibility)

Una API es compatible hacia atrás si las aplicaciones antiguas pueden seguir interactuando con ella incluso después de una actualización.

Cambios compatibles (seguros)

Tipo de cambio	Ejemplo	Descripción
Agregar campos opcionales	precio_descuento	Los clientes antiguos los ignoran sin romperse.
Añadir nuevos endpoints	/api/v2/recomendaciones	No afecta endpoints existentes.
Ampliar rango de valores permitidos	Nuevos estados válidos	Mejora funcional sin alterar semántica.

Cambios incompatibles (rompedores)

Tipo de cambio	Ejemplo	Consecuencia
Eliminar campos existentes	precio removido	Fallas en deserialización.
Cambiar nombre de propiedades	cliente_id → usuario_id	Contratos rotos.
Modificar formato o tipo de dato	string → int	Incompatibilidad inmediata.

Imagen 17. Compatibilidad hacia atrás



4) Deprecación controlada

Cuando una versión antigua debe retirarse, el proceso debe ser planificado y comunicado con antelación. Las cabeceras HTTP pueden informar a los clientes sobre su obsolescencia.

Deprecation: true
Sunset: Sat, 31 Dec 2025 23:59:59 GMT
Link: <https://api.empresa.com/docs/v2>; rel="successor-version"

Buenas prácticas para deprecación

1. Anunciar con al menos 6 meses de anticipación.
2. Publicar mensajes claros en la documentación y dashboard del API Gateway.
3. Ofrecer herramientas de migración o guías comparativas entre versiones.
4. Mantener registros de consumo para identificar clientes aún activos en versiones antiguas.

Imagen 18. Ciclo de vida de una API versionada



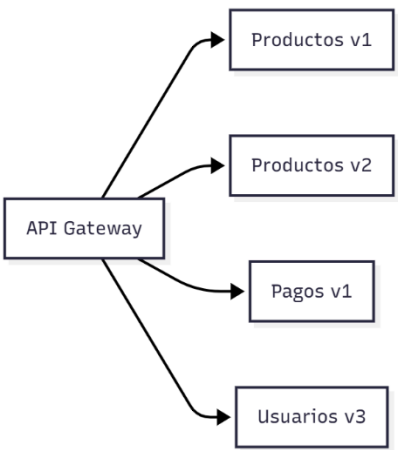
5) Versionamiento en microservicios

En arquitecturas distribuidas, el versionamiento debe aplicarse por servicio, ya que cada microservicio evoluciona de forma independiente.

Recomendaciones

- Mantener contratos OpenAPI independientes por microservicio.
- Evitar dependencias fuertes entre versiones (v1 → v2).
- Usar API Gateway para enrutar peticiones según versión.
- Implementar testing contractual (Pact, Postman) para validar compatibilidad.

Imagen 19. Versionamiento en ecosistema de microservicios



Resultado: coexistencia controlada de múltiples versiones de microservicios bajo un mismo Gateway, con reglas de enrutamiento y seguridad diferenciadas.

6) Tabla comparativa de estrategias

Estrategia	Visibilidad	Dificultad	Uso recomendado
En URL	Alta	Baja	APIs públicas y documentadas
En cabecera	Media	Media	APIs internas o empresariales
Por parámetro	Baja	Baja	Prototipos y pruebas
SemVer	Alta	Media	Microservicios y CI/CD

7) Buenas prácticas de versionamiento REST

- Documentar cada versión en un portal Swagger/Redoc.
- Mantener compatibilidad hacia atrás siempre que sea posible.
- Implementar tests de regresión antes de publicar nuevas versiones.
- Aplicar versionamiento semántico coherente con los cambios.
- Comunicar la deprecación mediante cabeceras y documentación oficial.
- Automatizar la generación de versiones en el pipeline de CI/CD.
- Centralizar el catálogo de APIs con sus respectivas versiones y consumidores activos.

2.5. Documentación dinámica con Swagger / Redoc

La documentación dinámica es un componente esencial del ciclo de vida de las APIs REST. En entornos empresariales y de microservicios, las herramientas como Swagger (OpenAPI) y Redoc permiten mantener una documentación actualizada, navegable y ejecutable, garantizando la transparencia, la interoperabilidad y la colaboración entre equipos de desarrollo.

1) Importancia de la documentación dinámica

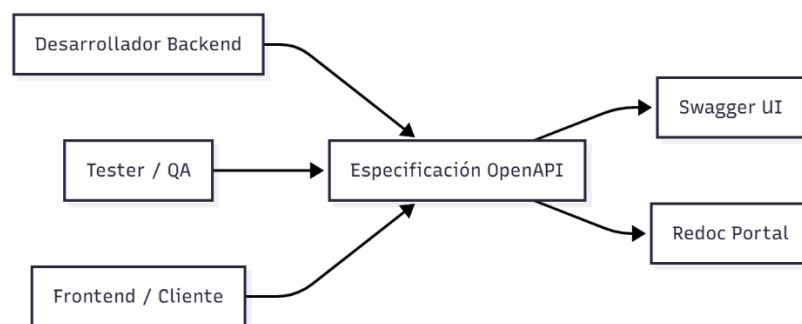
En una arquitectura moderna basada en APIs, la documentación no es un simple archivo adicional: es parte del contrato vivo entre el proveedor y el consumidor. Una API bien

documentada reduce el tiempo de integración, evita errores y acelera la adopción por parte de terceros.

Beneficios clave

- Autodescripción: los endpoints se explican mediante contratos OpenAPI legibles por humanos y máquinas.
- Interactividad: permite probar solicitudes directamente desde el navegador.
- Sincronización: el contrato se actualiza automáticamente con el código.
- Estandarización: fomenta un lenguaje común entre backend, frontend, QA y DevOps.

Imagen 20. Documentación como interfaz viva



Resultado: un único contrato técnico que sirve de fuente de verdad (Single Source of Truth) para toda la organización.

2) El estándar OpenAPI y su estructura

El estándar OpenAPI (anteriormente conocido como Swagger Specification) define el formato más usado para describir APIs REST. Su sintaxis se escribe en YAML o JSON, lo que facilita la legibilidad y la integración con pipelines automatizados.

Estructura básica

```
openapi: 3.0.3
info:
  title: API de Pedidos
  description: Servicio de gestión de pedidos en línea
  version: 1.0.0
servers:
  - url: https://api.empresa.com/v1
paths:
  /pedidos:
    get:
      summary: Lista los pedidos activos
      responses:
        '200':
          description: Operación exitosa
```

Componentes principales

Sección	Descripción
info	Metadatos: título, versión, descripción.
servers	Rutas base de despliegue (producción, QA, desarrollo).
paths	Endpoints y operaciones disponibles.
components	Definición de esquemas, respuestas, seguridad, parámetros.
security	Configuración de OAuth2, JWT o API Keys.

Imagen 21. Flujo de generación OpenAPI



3) Swagger UI: documentación interactiva

Swagger UI es la herramienta más popular para visualizar y probar APIs OpenAPI. Permite a los usuarios explorar endpoints, enviar solicitudes y observar respuestas sin usar herramientas externas.

Ventajas

- Navegación intuitiva por endpoints y métodos HTTP.
- Ejecución directa de pruebas desde el navegador.
- Integración con autenticación OAuth2 o API Key.
- Compatible con pipelines CI/CD y frameworks modernos (Spring Boot, FastAPI, NestJS).

Ejemplo de configuración (Spring Boot)

```
import io.swagger.v3.oas.annotations.OpenAPIDefinition;  
import io.swagger.v3.oas.annotations.info.Info;  
  
@OpenAPIDefinition(  
    info = @Info(title = "API de Productos", version = "1.0", description = "Documentación Swagger UI")  
)  
@SpringBootApplication  
public class ApiProductosApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(ApiProductosApplication.class, args);  
    }  
}
```

Imagen 22. Interfaz Swagger UI



Tabla 1. Ventajas y limitaciones de Swagger UI

Aspecto	Ventaja	Limitación
Interactividad	Permite probar endpoints en vivo	Requiere entorno accesible
Generación automática	Integra con frameworks	Puede exponer endpoints sensibles
Legibilidad	Intuitiva y visual	No ideal para publicación externa

4) Redoc: documentación elegante y corporativa

Redoc es una alternativa moderna enfocada en la experiencia de lectura y presentación profesional. Genera documentación estática o dinámica a partir de un archivo OpenAPI.

Características principales

- Visualización jerárquica y organizada.
- Integración de Markdown para notas técnicas.
- Soporte para temas personalizados y branding corporativo.
- Despliegue fácil en GitHub Pages, Nginx o portales de API.

Ejemplo de uso

```
npx redoc-cli bundle openapi.yaml -o index.html
```

Esto genera un sitio HTML navegable con toda la documentación de la API.

Imagen 23. Flujo Redoc



Comparativa entre Swagger y Redoc

Característica	Swagger UI	Redoc
Interactividad	Alta	Media
Estilo visual	Técnico	Elegante y corporativo
Soporte Markdown	Parcial	Completo
Integración CI/CD	Sí	Sí
Ideal para	Pruebas de desarrollo	Publicación oficial

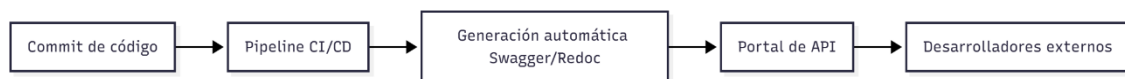
5) Automatización en CI/CD y portales de API

La documentación debe generarse y actualizarse automáticamente cada vez que se despliega una nueva versión de la API. Esto se logra integrando Swagger/Redoc en el pipeline de CI/CD.

Ejemplo de pipeline (GitHub Actions)

```
name: Deploy API Docs
on:
  push:
    branches: [ main ]
jobs:
  build-docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Generar documentación con Redoc
        run: npx redoc-cli bundle openapi.yaml -o docs/index.html
      - name: Publicar en GitHub Pages
        uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${ secrets.GITHUB_TOKEN }
          publish_dir: ./docs
```

Imagen 24. Flujo CI/CD de documentación



Portales de API empresariales

En organizaciones grandes, la documentación suele publicarse en portales especializados:

Herramienta	Descripción
Kong Developer Portal	Portal colaborativo con control de acceso.
Apigee Developer Portal	Integración con políticas de seguridad y monetización.
WSO2 API Manager	Control de ciclo de vida, suscripción y análisis de tráfico.

Ventajas:

- Centralización de APIs corporativas.
- Control de acceso por roles.
- Versionamiento y métricas de consumo.
- Integración con Single Sign-On (SSO) y OAuth2.

6) Buenas prácticas para documentación de APIs

- Mantener la documentación sincronizada con el código mediante validación en CI/CD.
- Añadir ejemplos de request y response reales.

- Definir estándares visuales: colores, íconos, branding.
- Incluir notas de seguridad (scopes, tokens, roles).
- Publicar versiones archivadas de la documentación (/v1, /v2).
- Agregar metadatos útiles: contacto, licencias, términos de uso.
- Validar el archivo OpenAPI con herramientas como Spectral, Swagger Editor o Stoplight Studio.

7) Tabla resumen: Swagger vs Redoc vs Portales CI/CD

Herramienta	Tipo	Ventajas principales	Ideal para
Swagger UI	Dinámica interactiva	Pruebas directas, debugging	Entornos de desarrollo
Redoc	Estática elegante	Publicación profesional	Portales corporativos
Apigee / Kong / WSO2	Portal completo	Gestión, control y monitoreo	Ecosistemas empresariales